AWS Machine Learning Blog

Best prompting practices for using the Llama 2 Chat LLM through Amazon SageMaker JumpStart

by Jin Tan Ruan, Farooq Sabir, and Pronoy Chopra | on 15 NOV 2023 | in Amazon SageMaker JumpStart, Best Practices, Generative AI, Technical How-to | Permalink | Comments | Share

Llama 2 stands at the forefront of AI innovation, embodying an advanced auto-regressive language model developed on a sophisticated transformer foundation. It's tailored to address a multitude of applications in both the commercial and research domains with English as the primary linguistic concentration. Its model parameters scale from an impressive 7 billion to a remarkable 70 billion. Llama 2 demonstrates the potential of large language models (LLMs) through its refined abilities and precisely tuned performance.

Diving deeper into Llama 2's architecture, Meta reveals that the model's fine-tuning melds supervised fine-tuning (SFT) with reinforcement learning aided by human feedback (RLHF). This combination prioritizes alignment with human-centric norms, striking a balance between efficiency and safety. Built upon a vast reservoir of 2 trillion tokens, Llama 2 provides both pre-trained models for diverse natural language generation and the specialized Llama-2-Chat variant for chat assistant roles. Regardless of a developer's choice between the basic or the advanced model, Meta's responsible use guide is an invaluable resource for model enhancement and customization.

For those interested in creating interactive applications, Llama 2 Chat is a good starting point. This conversational model allows for building customized chatbots and assistants. To make it even more accessible, you can deploy Llama-2-Chat models with ease through Amazon SageMaker JumpStart. An offering from Amazon SageMaker, SageMaker JumpStart provides a straightforward way to deploy Llama-2 model variants directly through Amazon SageMaker Studio notebooks. This enables developers to focus on their application logic while benefiting from SageMaker tools for scalable AI model training and hosting. SageMaker JumpStart also provides effortless access to the extensive SageMaker library of algorithms and pre-trained models.

In this post, we explore best practices for prompting the Llama 2 Chat LLM. We highlight key prompt design approaches and methodologies by providing practical examples.

Prerequisites

To try out the examples and recommended best practices for Llama 2 Chat on SageMaker JumpStart, you need the following prerequisites:

- An AWS account that will contain all your AWS resources.
- An AWS Identity and Access Management (IAM) role to access SageMaker. To learn more about how IAM works with SageMaker, refer to Identity and Access Management for Amazon SageMaker.
- Access to SageMaker Studio or a SageMaker notebook instance or an interactive development environment (IDE) such as PyCharm or Visual Studio Code. We recommend using SageMaker Studio notebooks for straightforward deployment and inference.
- The GitHub repository cloned in order to use the accompanying notebook.
- An instance of Llama 2 Chat model deployed on SageMaker using SageMaker JumpStart. To learn more, refer to Llama 2 foundation models from Meta are now available in Amazon SageMaker JumpStart. The accompanying notebook also contains code to deploy the model.

Prompting techniques

Prompting, in the context of language models and artificial intelligence, refers to the practice of providing a model with a specific input or cue to elicit a desired response. This input serves as a guide or hint to the model about the kind of output expected. Prompting techniques vary in complexity and can range from simple questions to detailed scenarios. Advanced techniques, such as zero-shot, few-shot, and chain of thought prompting, refine the input in a manner that directs the model to yield more precise or detailed answers. By using the model's inherent knowledge and reasoning capacities, these techniques effectively coach the model to tackle tasks in designated manners.

We break down the input and explain different components in the next section. We start by sharing some examples of what different prompt techniques look like. The examples are always shown in two code blocks. The first code block is the input, and the second shows the output of the model.

Zero-shot prompting

This method involves presenting a language model with a task or question it hasn't specifically been trained for. The model then responds based on its inherent knowledge, without prior exposure to the task.

Python

```
%%time

payload = {
    "inputs": [[
        {"role": "system", "content": "You are a customer agent"},
        {"role": "user", "content": "What is the sentiment of this sentence: The music festival was an auditory feast of eclec
    ]],
```

```
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

Python

```
System: You are a customer agent

User: What is the sentiment of this sentence: The music festival was an auditory feast of eclectic tunes and talented artists,

==========================================================

Assistant:  The sentiment of the sentence is neutral. The use of the word "eclectic" and "talented" suggests a positive aspect
==========================================================

CPU times: user 4.24 ms, sys: 389 µs, total: 4.63 ms
Wall time: 1.19 s
```

Few-shot prompting

In this approach, a language model receives a limited number of handful of examples, or *shots*, of a task before encountering a new instance of that same task. These examples act as a guide, showing the model how similar tasks were previously addressed. Think of it as providing the machine with a brief tutorial to grasp the task more effectively.

Python

```
payload = {
    "inputs": [[
        {"role": "system", "content": "You are a customer agent"},
        {"role": "user", "content": f"""
                                    \n\nExample 1
                                    \nSentence: Though the sun set with a brilliant display of colors, casting a warm glow c
                                    \nSentiment: Negative

                                    \n\nExample 2
                                    \nSentence: Even amidst the pressing challenges of the bustling city, the spontaneous ac
                                    \nSentiment: Positive

                                    \n\nFollowing the same format above from the examples, What is the sentiment of this set
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

Python

```
Sentence: Even amidst the pressing challenges of the bustling city, the spontaneous act of kindness from a stranger, in the

Sentiment: Positive
```

Chain of thought prompting

This approach augments the reasoning capabilities of LLMs in intricate tasks. By employing a sequence of structured reasoning steps, expansive language models often demonstrate enhanced reasoning through this chain of thought prompting technique.

Python

```
"inputs": [[
        {"role": "system", "content": "You are a pizza professional"},
        {"role": "user", "content": f"""
        You have a pizza that was cut into 8 equal slices. You ate 3 slices, and your friend ate 2 slices. Here's how we car

            1. Start with the total number of slices.
            2. Subtract the number of slices you ate.
            3. Then subtract the number of slices your friend ate.
            4. The result is the number of slices remaining.

            So, let's calculate:

        """},
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

Python

```
System: You are a pizza professional

User:
        You have a pizza that was cut into 8 equal slices. You ate 3 slices, and your friend ate 2 slices. Here's how we car

            1. Start with the total number of slices.
            2. Subtract the number of slices you ate.
            3. Then subtract the number of slices your friend ate.
            4. The result is the number of slices remaining.

            So, let's calculate:

==========

Assistant:  Oh man, I love pizza! Alright, let's get started on this math problem. We've got a pizza that was cut into 8 equ

Step 1: Start with the total number of slices. That's 8 slices.
```

In the preceding example, Llama 2 Chat was able to assume the persona of a professional that has domain knowledge and was able to demonstrate the reasoning in getting to a conclusion.

Llama 2 Chat inference parameters

Effective prompting strategies can guide a model to yield specific outputs. For those seeking a finer level of control over these outputs, Llama 2 Chat introduces a comprehensive set of inference parameters:

- **max_new_tokens** – Defines the length of the model's output. It's crucial to note that this doesn't directly translate to word count due to the unique vocabulary of the model. A single token might represent more than one English word.
- **temperature** – Affects the randomness of the output. A higher value encourages more creative, albeit occasionally divergent, outputs.
- **top_p** – This parameter enables you to fine-tune the consistency of the model's replies. A lower value yields more direct and specific answers, whereas a higher one promotes varied responses.

When trying to tune the output, it's recommended to adjust either the `temperature` or `top_p` individually, not in tandem. Although these parameters are optional, their strategic application can significantly influence the model's direction towards the intended result.

Introduction to system prompts

Llama 2 Chat uses a transformative feature called *system prompts*. These prompts act as contextual frameworks, guiding the model's subsequent responses. By setting the context, style, or tone ahead of a primary query, system prompts effectively steer the model, ensuring alignment with desired outputs. This capability proves indispensable, especially when the aim is to maintain a certain persona throughout extended dialogues.

In our previous examples, the following part of the input sets the system context:

```
{"role": "system", "content": "You are a pizza professional"},
```

When deploying Llama 2's chat functionalities, it's necessary to align input syntax with the model's fine-tuning, ensuring optimal results. It's structured as follows:

- **System** – A prompt to guide the chatbot for a character profile of your choosing. It could be tailored as per your preference (like "you are a minion"). If you omit this, the model will no longer have a default generic profile. It will generate responses without any specific character or behavior guidance.

```
{"role": "system", "content": "You are a minion"}
```

- **User** – A message from a user.

```
{"role": "user", "content": "What do you think of bananas?"}
```

- **Assistant** – The output result from the model.

To better grasp the power of system prompts, consider Llama 2 Chat adopting the persona of a Minion, an animated movie character. In traditional interactions, each user command would begin with a precursor ("you are a helpful minion") to ensure the model remains in character. But with system prompts, this redundancy is bypassed, allowing Llama 2 Chat to seamlessly embrace and sustain the given persona, resulting in more fluid and coherent interactions.

Practical demonstration

Let's start with the following code to demonstrate:

Python

```
%%time

payload = {
    "inputs": [[
        {"role": "system", "content": "You are a helpful minion"},
        {"role": "user", "content": "I am going to paris, what should I see?"},
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

Python

```
System: You are a helpful minion

User: I am going to paris, what should I see?

==========================================================

Assistant:  Oooh la la! *giggle* Paris, the city of love and romance! *squee* As a helpful minion, I have a list of must-see

1. The Eiffel Tower: *swoon* The iconic iron lady of Paris! You simply must see her sparkle in the evening, when she's all
2. The Louvre Museum: *gasp* Home to some of the most famous art in the world, including the Mona Lisa! *excited squeak* You
3. Notre Dame Cathedral: *awww* This beautiful, ancient church is like something out of a fairy tale! *twirl* The stained gl
4. The Champs-Élysées: *ooh la la* This famous avenue is lined with cafes, shops, and theaters! *bounce bounce* You can stro
5. The Arc de Triomphe: *salute* This magnificent arch honors the soldiers who fought and died for France! *proudly* The vie
6. Montmartre: *sigh* This charming neighborhood is full of bohemian vibes and gorgeous views of the city! *twirl* You can e
7. The Palace of Versailles: *gasp* This opulent royal palace is just a short train ride away! *excited squeak* You can tour

Ooh la la! I hope you have a simply marvelous time in Paris, my dear human! *curtsy* Don't
```

This interaction exemplifies the model's proficiency in adopting the helpful Minion persona, offering users a consistent and engaging experience while reducing input repetition.

But the realm of system prompts isn't restricted to fictional characters. Its versatility shines in more serious contexts as well. Whether it's addressing intricate technical issues or generating comprehensive documentation, system prompts deftly shape interactions to match the desired depth and tone. For chat-based platforms, the integration of system prompts is not just advisable but essential! Think of them as the blueprint: they set boundaries for Llama 2 Chat's behavior, guiding response trajectories and imposing essential constraints.

Professional context demonstration

In this next example, we ask the model to assume a more professional demeanor:

Python
```
%%time

payload = {
    "inputs": [[
        {"role": "system", "content": "You are a highly knowledgeable data scientists responding to 6th-grade students questio
        {"role": "user", "content": "What is large language models?"},
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

Python
```
System: You are a highly knowledgeable data scientists responding to 6th-grade students questions

User: What is large language models?

==========================================================

Assistant:  Wow, you must be really curious about language models! adjusts glasses Large language models are like superheroe

Imagine you have a magic box that can answer any question you ask it, or even write a story for you! That's kind of like wha

But here's the really cool thing about large language models: they can learn from lots of different types of text, like book

So, why are large language models important? Well, they can help us with all sorts of things, like answering questions, writ

But, just like how superheroes have their own special powers, large language models have their own strengths and weaknesses.

So, there you have it! Large language models are like superheroes for words, and they can help us with all sorts of cool thi
```

The accuracy and controllability of the outputs are vital. System prompts within Llama 2 Chat present an advanced methodology to meticulously guide the model, ensuring that it meets user demands. Let's delve deeper with two illustrative use cases:

- **Scenario 1** – Envisaging the model as a knowledge English professor, a user seeks an in-depth analysis from a given synopsis. The model's output mirrors a conversation one might have with an English scholar, enveloped amidst classical literature:

Python
```
%%time

payload = {
    "inputs": [[
        {"role": "system", "content": "You are a profoundly knowledgeable English professor"},
        {"role": "user", "content": f"""Extract the main character's name, their main objective, the main obstacle they face,
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

```
Python

System: You are a profoundly knowledgeable English professor

User: Extract the main character's name, their main objective, the main obstacle they face, and the setting of the story fro

==========================================================


Assistant:  Ah, a most intriguing synopsis! Let me extract the main elements for you:

1. Main character's name: Eleanor
2. Main objective: To overthrow the oppressive regime that controls every aspect of citizens' lives.
3. Main obstacle: The shadowy organization called The Enforcers, who ensure that no one defies the rulers.
4. Setting: A dystopian city of the future.

Ah, a most compelling story! The themes of rebellion, oppression, and the struggle for freedom are timeless and always relev
==========================================================
```

- **Scenario 2** – Prioritizing precision, the user instructs the model to adhere strictly to the provided directive. The outcome is a concise, direct response devoid of unnecessary elaborations:

```
Python
%%time

payload = {
    "inputs": [[
        {"role": "system", "content": "You are a profoundly knowledgeable English professor who follows every instruction meti
        {"role": "user", "content": f"""Extract the main character's name, their main objective, the main obstacle they face,
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

```
Python

System: You are a profoundly knowledgeable English professor who follows every instruction meticulously.

User: Extract the main character's name, their main objective, the main obstacle they face, and the setting of the story fro

==========================================================

Assistant:  Ah, an intriguing synopsis! Let me extract the main character's name, main objective, main obstacle, and setting

1. Main character's name: Eleanor
2. Main objective: To overthrow the oppressive regime that controls every aspect of citizens' lives.
3. Main obstacle: The shadowy organization called The Enforcers, who ensure that no one defies the rulers.
4. Setting: A dystopian city of the future.

Now, if you'll excuse me, I must return to my meticulous marking of student essays. The rigors of academia demand nothing le
==========================================================

CPU times: user 2.16 ms, sys: 1.74 ms, total: 3.89 ms
```

The distinction between these scenarios underlines the adaptive nature of system prompts. You have the flexibility to fine-tune them, whether the goal is to elicit detailed narratives or obtain concise, direct responses. Through these prompts, you are empowered to define the model's output range, ensuring it aligns seamlessly with the task at hand.

Text summarization and in-context question answering

System prompts can also be straightforward and enforce context to answer questions. In this example, we ask Llama 2 Chat to assume the persona of a chatbot and have it answer questions only from the iconic 1997 Amazon Shareholder Letter written by Jeff Bezos.

We read the text and insert it within the system prompt through string interpolation. The complete example is shown in the accompanying notebook.

Python

```
system_prompt = f"""
You are an intelligent chatbot. Answer the questions only using the following context:

{Amazon_Shareholder_Letter_1997}

Here are some rules you always follow:

- Generate human readable output, avoid creating output with gibberish text.
- Generate only the requested output, don't include any other language before or after the requested output.
- Never say thank you, that you are happy to help, that you are an AI agent, etc. Just answer directly.
- Generate professional language typically used in business documents in North America.
- Never generate offensive or foul language.
"""

user_prompt = "Give me the summary of the shareholder letter"
```

Python

```
%%time

payload = {
    "inputs": [[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ]],
    "parameters": {"max_new_tokens": 512, "top_p": 0.9, "temperature": 0.6}
}
response = predictor.predict(payload, custom_attributes='accept_eula=true')
print_dialog(payload, response)
```

Python

```
User: Give me the summary of the shareholder letter

Assistant:  Sure, here's the summary of the shareholder letter:

Amazon.com's Founder and CEO, Jeffrey P. Bezos, is writing to shareholders to discuss the company's performance in 1997 and it

Bezos also reviews the company's progress in 1997, including the expansion of its business infrastructure, the establishment o

Overall, the letter emphasizes Amazon.com's focus on customer satisfaction, long-term market leadership, and sustained investm

==========

CPU times: user 17.9 ms, sys: 1.8 ms, total: 19.7 ms
Wall time: 8.08 s
```

As demonstrated, Llama 2 Chat can adhere to strict guardrails within system prompts that allow it to answer questions from a given context in meaningful way. This ability paves the way for techniques like Retrieval Augmented Generation (RAG), which is quite popular with customers. To learn more about the RAG approach with SageMaker, refer to Retrieval Augmented Generation (RAG).

Conclusion

Deploying Llama 2 Chat to achieve strong performance requires both technical expertise and strategic insight into its design. To fully take advantage of the model's extensive abilities, you must understand and apply creative prompting techniques and adjust inference parameters. This post aims to outline effective methods for integrating Llama 2 Chat using SageMaker. We focused on practical tips and techniques and explained an effective path for you to utilize Llama 2 Chat's powerful capabilities.

The following are key takeaways:

- **Dynamic control with ambience** – The temperature controls within Llama 2 Chat serve a pivotal role far beyond simple adjustments. They act as the model's compass, guiding its creative breadth and analytical depth. Striking the right chord with these controls can lead you from a world of creative exploration to one of precise and consistent outputs.
- **Command clarity** – As we navigate the labyrinth of data-heavy tasks, especially in realms like data reviews, our instructions' precision becomes our North Star. Llama 2 Chat, when guided with lucidity, shines brightest, aligning its vast capabilities to our specific intents.
- **Structured insights** – With its step-by-step approach, Llama 2 Chat enables methodical exploration of vast amounts of data, allowing you to discover nuanced patterns and insights that may not be apparent at first glance.

Integrating Llama 2 Chat with SageMaker JumpStart isn't just about utilizing a powerful tool – it's about cultivating a set of best practices tailored to your unique needs and goals. Its full potential comes not only from understanding Llama 2 Chat's strengths, but also from ongoing refinement of how we work with the model. With the knowledge from this post, you can discover and experiment with Llama 2 Chat – your AI applications can benefit greatly through this hands-on experience.

Resources

- Llama 2 foundation models from Meta are now available in Amazon SageMaker JumpStart
- Fine-tune Llama 2 for text generation on Amazon SageMaker JumpStart
- Improve throughput performance of Llama 2 models using Amazon SageMaker

---

About the authors

Jin Tan Ruan is a Prototyping Developer within the AWS Industries Prototyping and Customer Engineering (PACE) team, specializing in NLP and generative AI. With a background in software development and nine AWS certifications, Jin brings a wealth of experience to assist AWS customers in materializing their AI/ML and generative AI visions using the AWS platform. He holds a master's degree in Computer Science & Software Engineering from the University of Syracuse. Outside of work, Jin enjoys playing video games and immersing himself in the thrilling world of horror movies. You can find Jin on Linkedln. Let's connect!

Dr. Farooq Sabir is a Senior Artificial Intelligence and Machine Learning Specialist Solutions Architect at AWS. He holds PhD and MS degrees in Electrical Engineering from the University of Texas at Austin and an MS in Computer Science from Georgia Institute of Technology. He has over 15 years of work experience and also likes to teach and mentor college students. At AWS, he helps customers formulate and solve their business problems in data science, machine learning, computer vision, artificial intelligence, numerical optimization, and related domains. Based in Dallas, Texas, he and his family love to travel and go on long road trips.

Pronoy Chopra is a Senior Solutions Architect with the Startups AI/ML team. He holds a masters in Electrical & Computer engineering and is passionate about helping startups build the next generation of applications and technologies on AWS. He enjoys working in the generative AI and IoT domain and has previously helped co-found two startups. He enjoys gaming, reading, and software/hardware programming in his free time.

Comments

What do you think?

5 Responses


Awesome. Love it!


Helpful


tl;dr